

فاز ۲ - بررسی مدل با استفاده از SPIN

ترکیب مدل با فایل ویژگی

از آنجا که SPIN نیاز دارد مدل و فرمول LTL در یک فایل قرار داشته باشند، ابتدا باید فایل ویژگی تولیدشده با فایل output.pml ادغام شود تا امکان بررسی آن فراهم گردد. این کار با یک عمل ساده‌ی الحاق فایل‌ها انجام می‌شود.
لینوکس / macOS

```
cat output.pml properties.ltl > merged.pml
```

ویندوز (cmd)

```
copy /b output.pml+properties.ltl merged.pml
```

سپس فایل حاصل، یعنی merged.pml همانند حالت معمول به SPIN داده می‌شود. در فایل properties.ltl تمامی ویژگی‌ها وجود دارد و در صورت نیاز ساخته می‌شود (مانند تعداد لوپ‌های در کد و...). از این قسمت به بعد، در تمام مثال‌ها تنها دستور cat نمایش داده شده است. در ویندوز کافی است آن را با دستور معادل copy /b که در بالا آمده است جایگزین کنید.

ویژگی‌های تولیدشده برای بررسی

safety

```
ltl safety {  
    [] (!divByZero)  
}
```

این یک ویژگی ایمنی (Safety) است. این ویژگی بیان می‌کند که یک وضعیت نامطلوب، یعنی برقرار شدن divByZero نباید در هیچ نقطه‌ای از اجرای برنامه رخ دهد. SPIN با جست‌وجوی تمام حالت‌های قابل دسترس بررسی می‌کند که آیا چنین وضعیتی قابل دستیابی است یا خیر. اگر هیچ حالتی یافت نشود، این ویژگی برقرار است.

liveness

```
ltl liveness_i {  
    [] (main@inLoop_i -> <>main@exitLoop_i)  
}
```

این یک ویژگی زنده‌بودن (Liveness) است که برای هر حلقه یک بار تولید می‌شود (که در آن i شماره حلقه است). این ویژگی بیان می‌کند که هر زمان اجرای برنامه وارد یک حلقه شود، باید در نهایت از آن خارج گردد. SPIN با جست‌وجوی مسیرهایی که وارد حلقه می‌شوند اما هرگز به برچسب خروج آن نمی‌رسند، این ویژگی را بررسی می‌کند. اگر چنین مسیری وجود نداشته باشد، ویژگی برقرار است.

reachability

```
ltl reachability {  
    <>endReached  
}
```

این یک ویژگی دسترس‌پذیری (Reachability) است. این ویژگی بیان می‌کند که برنامه باید در نهایت به حالت پایانی خود برسد؛ حالتی که با برقرار شدن endReached مشخص می‌شود. SPIN با جست‌وجوی مسیرهایی که به این حالت منتهی می‌شوند، این ویژگی را بررسی می‌کند. اگر حداقل یک مسیر به این حالت برسد، ویژگی برقرار است.

invariant

```
۱  ltl invariant {  
۲      [] (condition)  
۳  }
```

این یک ویژگی ناوردا (Invariant) است. این ویژگی بیان می‌کند که یک شرط مشخص باید در تمام حالت‌های اجرای برنامه برقرار باشد. SPIN با جست‌وجوی هر حالت قابل دسترس که در آن این شرط برقرار نباشد، این ویژگی را بررسی می‌کند. اگر چنین حالتی یافت نشود، ناوردا برقرار است.

مثال‌های بررسی

مثال ۱ - بررسی موفق

کد Hash

```
۱  adad x = 10;
۲  adad y = 2;
۳  adad z = x / y;
۴  ta (x > 0) {
۵      x = x - 1;
۶  }
```

کد Promela تولیدشده (output.pml)

```
۱  /* Generated Promela model from Hash */
۲
۳  bool divByZero = false;
۴  bool outOfBound = false;
۵  bool nullPointer = false;
۶  bool noPermission = false;
۷  bool endReached = false;
۸  int x;
۹  int y;
۱۰ int z;
۱۱
۱۲ active proctype main() {
۱۳     x = 10;
۱۴     y = 2;
۱۵     if
۱۶     :: (y == 0) ->
۱۷         divByZero = true;
۱۸         goto endReached_label;
۱۹     :: else ->
۲۰         skip;
۲۱     fi;
۲۲     z = x/y;
۲۳     loop_start_1:
۲۴     do
۲۵     :: (x>0) ->
۲۶         inLoop_1:
۲۷         x = x-1;
۲۸         ;
۲۹
۳۰     :: else -> break
۳۱     od;
۳۲     exitLoop_1:
۳۳     skip;
۳۴
۳۵     endReached = true;
۳۶     endReached_label:
۳۷     skip;
۳۸ }
```

```

۱ cat output.pml properties.ltl > merged.pml
۲ spin -search -ltl safety merged.pml

```

Output:

```

۱ (Spin Version 2.5.6 -- 6 December 2019)
۲ + Partial Order Reduction
۳
۴ Full statespace search for:
۵ never claim + (safety)
۶ assertion violations + (if within scope of claim)
۷ cycle checks - (disabled by -DSAFETY)
۸ invalid end states - (disabled by never claim)
۹
۱۰ State-vector 28 byte, depth reached 61, errors: 0
۱۱ 31 states, stored
۱۲ 1 states, matched
۱۳ 32 transitions (= stored+matched)
۱۴ 0 atomic steps
۱۵ hash conflicts: 0 (resolved)
۱۶
۱۷ Stats on memory usage (in Megabytes):
۱۸ 002.0 equivalent memory usage for states (stored*(State-vector + overhead))
۱۹ 290.0 actual memory usage for states
۲۰ 000.128 memory used for hash table (-w24)
۲۱ 534.0 memory used for DFS stack (-m10000)
۲۲ 730.128 total actual memory usage
۲۳
۲۴
۲۵ unreachable in proctype main
۲۶ merged.pml:17, state 4, "divByZero = 1"
۲۷ (1 of 21 states)
۲۸ unreachable in claim safety
۲۹ _spin_nvr.tmp:8, state 10, "-end-"
۳۰ (1 of 10 states)

```

توضیح

در این بررسی، SPIN مدل تولیدشده را همراه با ویژگی ایمنی تحلیل کرده و تمام حالت‌های قابل دسترس را پیمایش می‌کند. در طول بررسی، مشخص می‌شود که آیا متغیر `divByZero` در هر مرحله‌ای از اجرا می‌تواند مقدار درست بگیرد یا خیر. از آنجا که مقسوم‌علیه با مقداری غیرصفر مقداردهی اولیه شده است، وضعیت خطا هرگز قابل دسترس نیست. بنابراین بررسی بدون گزارش هیچ‌گونه تخلفی پایان می‌یابد و نشان می‌دهد که ویژگی ایمنی در تمام اجراهای ممکن برقرار است.

```
spin -search -ltl liveness_1 merged.pml
```

Output

```
(Spin Version 2.5.6 -- 6 December 2019)
+ Partial Order Reduction

Full statespace search for:
never claim + (liveness_1)
assertion violations + (if within scope of claim)
acceptance cycles + (fairness disabled)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 61, errors: 0
51 states, stored (71 visited)
28 states, matched
99 transitions (= visited+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
003.0 equivalent memory usage for states (stored*(State-vector + overhead))
289.0 actual memory usage for states
000.128 memory used for hash table (-w24)
534.0 memory used for DFS stack (-m10000)
730.128 total actual memory usage

unreached in proctype main
merged.pml:17, state 4, "divByZero = 1"
(1 of 21 states)
unreached in claim liveness_1
_spin_nvr.tmp:10, state 13, "-end-"
(1 of 13 states)
```

توضیح

در این بررسی، SPIN ابتدا فرمول LTL را به یک Claim Never داخلی تبدیل کرده و سپس آن را روی مدل تولیدشده اعمال می‌کند. در ادامه فضای حالت را جست‌وجو می‌کند تا مشخص شود آیا مسیری وجود دارد که وارد حلقه شود اما هرگز به خروج آن نرسد یا خیر. از آنجا که مقدار x در هر تکرار کاهش می‌یابد، حلقه در نهایت خاتمه پیدا می‌کند. بنابراین هیچ مسیر ناقصی یافت نشده و ویژگی زنده‌بودن برقرار است.

```
spin -search -ltl reachability merged.pml
```

Output

```
(Spin Version 2.5.6 -- 6 December 2019)
+ Partial Order Reduction

Full statespace search for:
never claim + (reachability)
assertion violations + (if within scope of claim)
acceptance cycles + (fairness disabled)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 56, errors: 0
29 states, stored (58 visited)
27 states, matched
85 transitions (= visited+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
002.0 equivalent memory usage for states (stored*(State-vector + overhead))
290.0 actual memory usage for states
000.128 memory used for hash table (-w24)
534.0 memory used for DFS stack (-m10000)
730.128 total actual memory usage

unreached in proctype main
merged.pml:17, state 4, "divByZero = 1"
merged.pml:38, state 21, "-end-"
(2 of 21 states)
unreached in claim reachability
_spin_nvr.tmp:6, state 6, "-end-"
(1 of 6 states)
```

توضیح

SPIN فضای حالت را بررسی می کند تا مشخص شود آیا برنامه می تواند به حالت پایانی تعیین شده برسد یا خیر. نتیجه بررسی نشان می دهد که حداقل یک مسیر اجرایی وجود دارد که به حالت endReached منتهی می شود. از آنجا که هیچ تخلفی از ویژگی گزارش نشده است، نتیجه می گیریم که برنامه می تواند اجرای خود را به صورت عادی به پایان برساند.

بررسی بن بست

بررسی بن بست به هیچ ویژگی LTL وابسته نیست؛ بنابراین مستقیماً روی فایل output.pml انجام می‌شود و نیازی به ادغام با فایل ویژگی ندارد.

```
spin -search -deadlock output.pml
```

Output

```
proctype main
state 1 -(tr 3)-> state 2 [id 0 tp 2] [----G] output.pml:13 => x = 10
state 2 -(tr 4)-> state 8 [id 1 tp 2] [----G] output.pml:14 => y = 2
state 8 -(tr 5)-> state 4 [id 2 tp 2] [----G] output.pml:16 => ((y==0))
state 8 -(tr 2)-> state 7 [id 5 tp 2] [----G] output.pml:16 => else
state 4 -(tr 6)-> state 20 [id 3 tp 2] [----G] output.pml:17 => divByZero = 1
state 20 -(tr 1)-> state 21 [id 19 tp 2] [--e-L] output.pml:37 => (1)
state 21 -(tr 11)-> state 0 [id 20 tp 3500] [--e-L] output.pml:38 => -end-
state 7 -(tr 1)-> state 10 [id 6 tp 2] [----L] output.pml:20 => (1)
state 10 -(tr 7)-> state 15 [id 9 tp 2] [----G] output.pml:22 => z = (x/y)
state 15 -(tr 8)-> state 12 [id 10 tp 2] [----G] output.pml:25 => ((x>0))
state 15 -(tr 2)-> state 18 [id 12 tp 2] [----G] output.pml:25 => else
state 12 -(tr 9)-> state 15 [id 11 tp 2] [----G] output.pml:27 => x = (x-1)
state 18 -(tr 1)-> state 19 [id 17 tp 2] [----L] output.pml:33 => (1)
state 19 -(tr 10)-> state 20 [id 18 tp 2] [----G] output.pml:35 => endReached = 1

Transition Type: A=atomic; D=d_step; L=local; G=global
Source-State Labels: p=progress; e=end; a=accept;
Note: statement merging was used. Only the first
stmtnt executed in each merge sequence is shown
(use spin -a -o3 to disable statement merging)
```

توضیح

در بررسی بن بست، SPIN بدون استفاده از هیچ ویژگی LTL مدل تولید شده را تحلیل می‌کند. این ابزار تمام حالت‌های قابل دسترس را پیمایش کرده و بررسی می‌کند که آیا در هر مرحله حداقل یک انتقال قابل اجرا وجود دارد یا خیر. از آنجا که هیچ حالتی یافت نمی‌شود که اجرای برنامه در آن به طور دائمی متوقف شود، نتیجه بررسی نشان می‌دهد که مدل فاقد بن بست است.

```
spin -search -ltl invariant merged.pml
```

Output

```
(Spin Version 2.5.6 -- 6 December 2019)
+ Partial Order Reduction

Full statespace search for:
never claim + (invariant)
assertion violations + (if within scope of claim)
cycle checks - (disabled by -DSAFETY)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 61, errors: 0
31 states, stored
1 states, matched
32 transitions (= stored+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
002.0 equivalent memory usage for states (stored*(State-vector + overhead))
290.0 actual memory usage for states
000.128 memory used for hash table (-w24)
534.0 memory used for DFS stack (-m10000)
730.128 total actual memory usage

unreached in proctype main
merged.pml:17, state 4, "divByZero = 1"
(1 of 21 states)
unreached in claim invariant
_spin_nvr.tmp:8, state 10, "-end-"
(1 of 10 states)
```

توضیح

SPIN بررسی می‌کند که شرط ناوردا در تمام حالت‌های قابل دسترس برنامه برقرار باقی بماند. در طول جست‌وجو تلاش می‌شود حالتی پیدا شود که این شرط در آن نقض شده باشد. از آنجا که مقدار x فقط تا صفر کاهش یافته و سپس حلقه خاتمه می‌یابد، ناوردا در تمام اجرای برنامه برقرار باقی می‌ماند. بنابراین هیچ نقضی گزارش نمی‌شود.

مثال ۲ - نقض ویژگی ایمنی

کد Hash

```
۱ adad x = 10;
۲ adad y = 2;
۳ adad z = x / y;
۴ ta (x > 0) {
۵     x = x - 1;
۶ }
```

کد Promela تولیدشده (output.pml)

```
۱ /* Generated Promela model from Hash */
۲
۳ bool divByZero = false;
۴ bool outOfBound = false;
۵ bool nullPointer = false;
۶ bool noPermission = false;
۷ bool endReached = false;
۸ int x;
۹ int y;
۱۰ int z;
۱۱
۱۲ active proctype main() {
۱۳     x = 10;
۱۴     y = 0;
۱۵     if
۱۶     :: (y == 0) ->
۱۷         divByZero = true;
۱۸         goto endReached_label;
۱۹     :: else ->
۲۰         skip;
۲۱     fi;
۲۲     z = x/y;
۲۳     loop_start_1:
۲۴     do
۲۵     :: (x>0) ->
۲۶         inLoop_1:
۲۷         x = x-1;
۲۸         ;
۲۹
۳۰     :: else -> break
۳۱     od;
۳۲     exitLoop_1:
۳۳     skip;
۳۴
۳۵     endReached = true;
۳۶     endReached_label:
۳۷     skip;
۳۸ }
```

```

۱ cat output.pml properties.ltl > merged.pml
۲ spin -search -ltl safety merged.pml

```

Verification Output

```

۱ (Spin Version 2.5.6 -- 6 December 2019)
۲ Warning: Search not completed
۳ + Partial Order Reduction
۴
۵ Full statespace search for:
۶ never claim + (safety)
۷ assertion violations + (if within scope of claim)
۸ cycle checks - (disabled by -DSAFETY)
۹ invalid end states - (disabled by never claim)
۱۰
۱۱ State-vector 28 byte, depth reached 8, errors: 1
۱۲ 5 states, stored
۱۳ 0 states, matched
۱۴ 5 transitions (= stored+matched)
۱۵ 0 atomic steps
۱۶ hash conflicts: 0 (resolved)
۱۷
۱۸ Stats on memory usage (in Megabytes):
۱۹ 000.0 equivalent memory usage for states (stored*(State-vector + overhead))
۲۰ 290.0 actual memory usage for states
۲۱ 000.128 memory used for hash table (-w24)
۲۲ 534.0 memory used for DFS stack (-m10000)
۲۳ 730.128 total actual memory usage

```

توضیح

در این مثال، مقسوم علیه پیش از انجام عمل تقسیم با مقدار صفر مقداردهی اولیه شده است. در نتیجه، مدل تولید شده متغیر `divByZero` را فعال می کند که باعث نقض ویژگی ایمنی می شود. SPIN این مثال نقض را در هنگام جست و جوی فضای حالت شناسایی کرده و آن را به عنوان تخلف از ویژگی گزارش می کند. از آنجا که یک اجرای ناقص پیدا شده است، فرایند بررسی با گزارش خطا پایان می یابد.

مثال ۳ - نقض ویژگی زنده بودن

کد Hash

```
۱ adad x = 5;
۲ ta (x > 0) {
۳     edame;
۴ }
```

کد Promela تولیدشده (output.pml)

```
۱ /* Generated Promela model from Hash */
۲
۳ bool divByZero = false;
۴ bool outOfBound = false;
۵ bool nullPointer = false;
۶ bool noPermission = false;
۷ bool endReached = false;
۸ int x;
۹
۱۰ active proctype main() {
۱۱     x = 5;
۱۲     loop_start_1:
۱۳     do
۱۴     :: (x>0) ->
۱۵         inLoop_1:
۱۶         goto loop_start_1;
۱۷
۱۸     :: else -> break
۱۹     od;
۲۰     exitLoop_1:
۲۱     skip;
۲۲
۲۳     endReached = true;
۲۴     endReached_label:
۲۵     skip;
۲۶ }
```

```

۱ cat output.pml properties.ltl > merged.pml
۲ spin -search -ltl liveness_1 merged.pml

```

Verification Output

```

۱ (Spin Version 2.5.6 -- 6 December 2019)
۲ Warning: Search not completed
۳ + Partial Order Reduction
۴
۵ Full statespace search for:
۶ never claim + (liveness_1)
۷ assertion violations + (if within scope of claim)
۸ acceptance cycles + (fairness disabled)
۹ invalid end states - (disabled by never claim)
۱۰
۱۱ State-vector 28 byte, depth reached 9, errors: 1
۱۲ 5 states, stored
۱۳ 0 states, matched
۱۴ 5 transitions (= stored+matched)
۱۵ 0 atomic steps
۱۶ hash conflicts: 0 (resolved)
۱۷
۱۸ Stats on memory usage (in Megabytes):
۱۹ 000.0 equivalent memory usage for states (stored*(State-vector + overhead))
۲۰ 290.0 actual memory usage for states
۲۱ 000.128 memory used for hash table (-w24)
۲۲ 534.0 memory used for DFS stack (-m10000)
۲۳ 730.128 total actual memory usage

```

توضیح

در این بررسی، SPIN کنترل می‌کند که هر اجرایی که وارد حلقه می‌شود، در نهایت بتواند از آن خارج شود. در مدل تولیدشده، برنامه به‌طور مداوم به ابتدای حلقه بازمی‌گردد، بدون آنکه شرط حلقه تغییری کند. در نتیجه یک اجرای بی‌نهایت ایجاد می‌شود و SPIN می‌تواند نمونه‌ای پیدا کند که ویژگی زنده‌بودن را نقض می‌کند. بنابراین بررسی با گزارش خطا پایان می‌یابد.

مثال ۴ - نقض ویژگی دسترس پذیری

کد Hash

```
۱ adad x;  
۲ ta (dorost) {}
```

کد Promela تولیدشده (output.pml)

```
۱ /* Generated Promela model from Hash */  
۲  
۳ bool divByZero = false;  
۴ bool outOfBound = false;  
۵ bool nullPointer = false;  
۶ bool noPermission = false;  
۷ bool endReached = false;  
۸ int x;  
۹  
۱۰ active proctype main() {  
۱۱     loop_start_1:  
۱۲     do  
۱۳     :: (true) ->  
۱۴     inLoop_1:  
۱۵     skip;  
۱۶  
۱۷     :: else -> break  
۱۸     od;  
۱۹     exitLoop_1:  
۲۰     skip;  
۲۱  
۲۲     endReached = true;  
۲۳     endReached_label:  
۲۴     skip;  
۲۵ }
```

```

۱ cat output.pml properties.ltl > merged.pml
۲ spin -search -ltl reachability merged.pml

```

Verification Output

```

۱ (Spin Version 2.5.6 -- 6 December 2019)
۲ Warning: Search not completed
۳ + Partial Order Reduction
۴
۵ Full statespace search for:
۶ never claim + (reachability)
۷ assertion violations + (if within scope of claim)
۸ acceptance cycles + (fairness disabled)
۹ invalid end states - (disabled by never claim)
۱۰
۱۱ State-vector 20 byte, depth reached 3, errors: 1
۱۲ 2 states, stored
۱۳ 0 states, matched
۱۴ 2 transitions (= stored+matched)
۱۵ 0 atomic steps
۱۶ hash conflicts: 0 (resolved)
۱۷
۱۸ Stats on memory usage (in Megabytes):
۱۹ 000.0 equivalent memory usage for states (stored*(State-vector + overhead))
۲۰ 291.0 actual memory usage for states
۲۱ 000.128 memory used for hash table (-w24)
۲۲ 534.0 memory used for DFS stack (-m10000)
۲۳ 730.128 total actual memory usage

```

توضیح

شرط حلقه همواره برقرار است؛ بنابراین اجرای برنامه هرگز از حلقه خارج نشده و به انتهای برنامه نمی‌رسد. SPIN تمام حالت‌های قابل دسترس را بررسی کرده و تشخیص می‌دهد که هیچ مسیر اجرایی قادر به ارضای ویژگی دسترس‌پذیری نیست. در نتیجه، یک مثال نقض تولید می‌شود که نشان می‌دهد حالت پایانی غیرقابل دسترس است.

مثال ۵ - نقض ناورد

کد Hash

```
۱ adad x = 2;  
۲  
۳ ta (x >= 0) {  
۴     x = x - 1;  
۵ }
```

کد Promela تولیدشده (output.pml)

```
۱ /* Generated Promela model from Hash */  
۲  
۳ bool divByZero = false;  
۴ bool outOfBound = false;  
۵ bool nullPointer = false;  
۶ bool noPermission = false;  
۷ bool endReached = false;  
۸ int x;  
۹  
۱۰ active proctype main() {  
۱۱     x = 2;  
۱۲     loop_start_1:  
۱۳     do  
۱۴         :: (x>=0) ->  
۱۵             inLoop_1:  
۱۶                 x = x-1;  
۱۷                 ;  
۱۸  
۱۹         :: else -> break  
۲۰     od;  
۲۱     exitLoop_1:  
۲۲     skip;  
۲۳  
۲۴     endReached = true;  
۲۵     endReached_label:  
۲۶     skip;  
۲۷ }
```

```

۱ cat output.pml properties.ltl > merged.pml
۲ spin -search -ltl invariant merged.pml

```

Verification Output

```

۱ (Spin Version 2.5.6 -- 6 December 2019)
۲ Warning: Search not completed
۳ + Partial Order Reduction
۴
۵ Full statespace search for:
۶ never claim + (invariant)
۷ assertion violations + (if within scope of claim)
۸ cycle checks - (disabled by -DSAFETY)
۹ invalid end states - (disabled by never claim)
۱۰
۱۱ State-vector 28 byte, depth reached 14, errors: 1
۱۲ 8 states, stored
۱۳ 0 states, matched
۱۴ 8 transitions (= stored+matched)
۱۵ 0 atomic steps
۱۶ hash conflicts: 0 (resolved)
۱۷
۱۸ Stats on memory usage (in Megabytes):
۱۹ 000.0 equivalent memory usage for states (stored*(State-vector + overhead))
۲۰ 290.0 actual memory usage for states
۲۱ 000.128 memory used for hash table (-w24)
۲۲ 534.0 memory used for DFS stack (-m10000)
۲۳ 730.128 total actual memory usage

```

توضیح

در این مثال، ناوردا بیان می‌کند که مقدار متغیر x باید در تمام طول اجرای برنامه نامنفی باقی بماند. با این حال، حلقه تا زمانی اجرا می‌شود که شرط $x \geq 0$ برقرار باشد؛ در نتیجه پس از آخرین تکرار، مقدار x منفی می‌شود. SPIN این حالت قابل دسترس را شناسایی کرده و آن را به عنوان نقض ناوردا گزارش می‌کند. بنابراین بررسی با شکست مواجه می‌شود.